

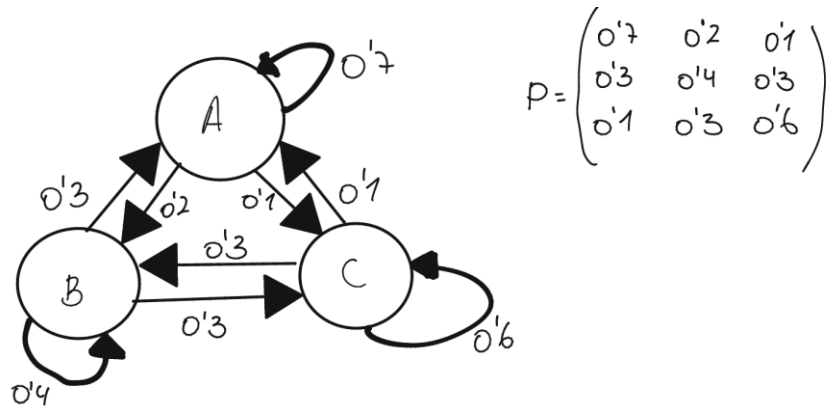
LAB 3 – INTELLIGENT DECISION-MAKING IN ROBOTICS

IGNACIO GONZÁLEZ MELÉNDEZ

NIA: 100522976

EXERCISE 1:

In this case the starting state is A, which has a probability of staying in the same state of 0.7, that means that in most cases it will be difficult to reach another state. Also, it is more likely to stay in the same state than change it. This can be seen in Figure 1.

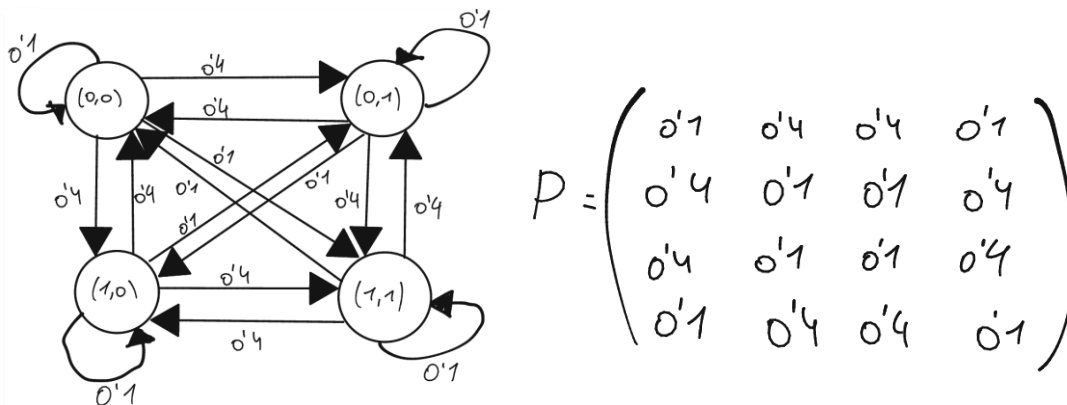


```
(venv) ignagonmell@DESKTOP-SVHOVNC:~/MARKOV$ python EXERCISE1.py
States visited: ['A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'C', 'C', 'B', 'A', 'B', 'A', 'A']
```

Figure 1

EXERCISE 2:

In the output of this exercise, Figure 2, we can see that the robot ended two times in a non-desired cell [(0,1) -> (1,0) & (1,1) -> (0,0)].



```
(venv) ignagonmell@DESKTOP-SVHOVNC:~/MARKOV$ python EXERCISE2.py
States visited: ['(1,1)', '(1,0)', '(0,0)', '(0,1)', '(0,1)', '(1,0)', '(1,1)', '(1,1)', '(0,1)', '(1,1)', '(0,0)']
```

Figure 2

EXERCISE 3:

Completed in EXERCISE3.py code.

EXERCISE 4:

In Figure 3, we can see the output in the terminal when running the code from exercise 3, we can see in “Optimal Policy” that the robot is trying to reach state (1,1) because it is where he gets a reward. And in “State Values” we can clearly see that those states closer to (1,1) get a better reward than state (0,0) because of its proximity to the reward.

```
(venv) ignagonmell@DESKTOP-SVHOVNC:~/MARKOV$ python EXERCISE3.py
Using the PolicyIteration method
Optimal Policy: (1, 1, 3, 3)
State Values: (7.8979440114486845, 8.764047442326078, 8.756252511448182, 8.764047442326078)
```

Figure 3

Now, state (1,0) gives the robot a penalization of -10, which means that the robot will try to avoid that state. This can be seen in Figure 4, the robot will go passing by state (0,1) in this case, meanwhile in the previous one it passed by state (1,0). Also, the rewards are affected by being reduced in a 40% approx.

```
(venv) ignagonmell@DESKTOP-SVHOVNC:~/MARKOV$ python EXERCISE4.py
Using the PolicyIteration method
Optimal Policy: (3, 1, 3, 3)
State Values: (4.765642917076287, 5.735721493440972, 5.72699078625369, 5.735721493440972)
```

Figure 4

EXERCISE 5:

In this final exercise, we have a 3x3 grid with a penalty of -1 in (1,1) and a reward of +1 in (2,2). Having said that, in Figure 4, output of EXERCISE5.py, we can see that the robot will try to avoid state (1,1) resulting in better “State Values” for those states near the goal. On the other hand, if the robot starts at (0,0) it will go DOWN to (1,0), then DOWN another time to state (2,0) and from here it will go two times to the RIGHT reaching (2,2). In this last state the robot will try to go DOWN getting away from penalty state.

```
(venv) ignagonmell@DESKTOP-SVHOVNC:~/MARKOV$ python EXERCISE5.py
Using the PolicyIteration method
Optimal Policy: (1, 3, 1, 1, 1, 1, 3, 3, 1)
State Values: (5.006060348928424, 5.624081256203538, 6.475473122110611, 5.624081256203538, 6.510836237280122, 7.481268372308802, 6.475473122110611, 7.481268372308802, 7.666529667912805)
```

Figure 5

EXERCISE 6:

In exercise 6, both (1,2) and (1,1) give the robot a penalty of -1. That's why we can see in Figure 6 that the “State Values” are decreased a little bit. However, the paths to reach the goal are still the same, including the path that starts at (0,0).

```
(venv) ignagonelli@DESKTOP-SVH0WNC:~/MARKOV$ python EXERCISE6.py
Using the PolicyIteration method
Optimal Policy: (1, 3, 1, 1, 1, 1, 3, 3, 1)
State Values: (4.37057682461102, 4.181252961725319, 4.713344614048643, 5.001203679502178, 5.68977207857121, 6.545508386733315, 5.7834042619929135, 6.684543676756662, 6.757545663264606)
```

Figure 6

Now, the penalty of states (1,1) and (1,2) has been increased to -10. Resulting in an output that can be seen in Figure 7. We can see that if we increase the penalty, the robot won't be able to reach the goal because the penalty is way bigger than the reward, giving negative rewards for all states. Also, the path to reach the goal starting at (0,0) is impossible because the best action in that state to maximize the expected cumulative reward is going upwards.

```
(venv) ignagonelli@DESKTOP-SVH0WNC:~/MARKOV$ python EXERCISE6.py
Using the PolicyIteration method
Optimal Policy: (0, 2, 2, 1, 1, 1, 1, 3, 1)
State Values: (-1.1877861547761488, -1.9477045661469048, -2.882022323544365, -1.7476381376011107, -2.20959252685284, -2.447700297773343, -1.0827652153829213, -1.6209727547013097, -2.022073481152568)
```

Figure 7

EXERCISE 7:

In this last case, just state (1,1) gives a penalty if -1 and the goal is (2,2) with a reward of +1. To avoid the robot hitting the walls, I decided to give the robot a penalty of -10 when the robot performs an action and stays at the same position in the states near the walls, for example going up in (0,0) and staying at the same position. Now, the robot will have the same path as the one in exercise 5 but when it is in position (2,2) instead of trying to go DOWN he will go UP in order to not hit the wall. Look at Figure 8.

```
(venv) ignagonelli@DESKTOP-SVH0WNC:~/MARKOV$ python EXERCISE7.py
Optimal Policy: (1, 3, 1, 1, 1, 1, 1, 3, 0)
State Values: (2.7539423081740297, 3.0939351857263775, 3.600441152717789, 3.093935185726379, 3.619369488878354, 4.16381566980252, 3.6004411527177904, 4.163815669802521, 3.81614361817587)
```

Figure 8

In this exercise, we could have given a penalty of -10 each time the robot performs an action and stays in the same position. However, we are trying to teach the robot not to hit the walls on purpose, this is by giving negative reward if it performs an action that is going to hit a wall, but if it hits a wall by accident I decided not to give a penalty because it is a random event where the robot wanted to go to a possible position but ended up hitting the wall because of slipping.